Projet Pluridisciplinaire

MCP Server Documentation

This document describes the implementation of the Industrial IoT MCP Server developed for Oil & Gas pipeline infrastructure monitoring. The server enables AI systems to interact with industrial infrastructure through the Model Context Protocol (MCP).

Technologies Used

Technology	Purpose
Python	Backend language
FastMCP	MCP framework
PostgreSQL	Industrial database
Psycopg2	Database driver
Python Dotenv	Environment variables

MCP Server Architecture

- Industrial infrastructure data is stored in PostgreSQL.
- FastMCP exposes industrial operations as MCP tools.
- AI systems communicate with the server through natural language prompts.
- The server executes SQL queries and returns industrial analytics.
- Triggers and views provide real-time industrial monitoring.

Implemented MCP Tools

get_machine_status

Returns operational information and health status of industrial machines.

get_sensor_value

Returns latest telemetry values from industrial sensors.

get_sensor_history

Returns historical telemetry measurements for a sensor.

list_active_alerts

Returns all active industrial alerts.

get_active_alerts_by_zone

Returns alerts filtered by industrial zone.

get_pipeline_info

Returns information about industrial pipelines.

get_station_status

Returns operational status of pumping and compression stations.

get_maintenance_history

Returns maintenance history for industrial machines.

update_machine_status

Updates operational status of machines.

get_machines_in_alarm

Returns all machines currently in alarm state.

resolve_alert

Resolves industrial alerts.

get_zone_overview

Provides global overview of industrial zones.

get_machine_availability

Analyzes reliability and availability of machines.

get_top_alerted_machines

Identifies machines generating the highest number of alerts.

get_temperature_anomalies

Detects abnormal industrial temperatures.

get_pressure_trends

Analyzes pressure telemetry trends.

get_pipeline_flow_rate

Analyzes pipeline throughput and flow rate.

compare_zones

Compares operational performance between industrial zones.

get_machine_health_score

Calculates AI-style health score for machines.

emergency_shutdown

Performs emergency stop operations.

create_maintenance_ticket

Creates industrial maintenance operations.

run_query

Executes custom SQL SELECT queries securely.

schema_overview

Provides textual description of the industrial schema.

AI Prompts and Use Cases

1. Show all active industrial alerts.

2. What is the latest pressure reading for sensor C1?

3. Which machines are currently in alarm?

4. Show all stations currently operational.

5. Which machines have the worst health score?

6. Detect abnormal temperatures in the infrastructure.

7. Which machines generate the most alerts?

8. Show machines with low availability.

9. Compare operational performance between zones.

10. Analyze pressure trends across the pipelines.

11. Show average pipeline flow rates.

12. Give me a summary of the industrial infrastructure.

13. Update machine M1 status to Maintenance.

14. Emergency shutdown machine M2.

15. Create a corrective maintenance ticket for machine M1.

16. Resolve alert 3.

17. Which zone is the most unstable?

18. Which pipeline infrastructure requires urgent maintenance?

19. Summarize the operational state of the oil and gas infrastructure.

20. Which industrial assets are at highest risk?

21. Show machine M1 status.

22. Show pressure history for sensor C1.

23. Show active alerts in Zone A.

24. Show information about pipeline P1.

25. Show maintenance history for machine M2.

26. Show machines with low availability.

27. Which machines are unstable?

28. Analyze pressure trends.

29. Show average pipeline flow rates.

30. Emergency shutdown machine M1.

31. Run a query to show all machines in maintenance.

Security Features

- Database credentials are stored using environment variables.
- The run_query tool only accepts SELECT statements.
- Critical operations are isolated into dedicated MCP tools.
- Operational commands are validated before execution.

Conclusion

The developed MCP Server successfully demonstrates the integration of Artificial Intelligence with Industrial IoT infrastructure using the Model Context Protocol (MCP). The server provides real-time monitoring, predictive maintenance support, industrial analytics, alert management, and operational control for Oil & Gas pipeline infrastructures.